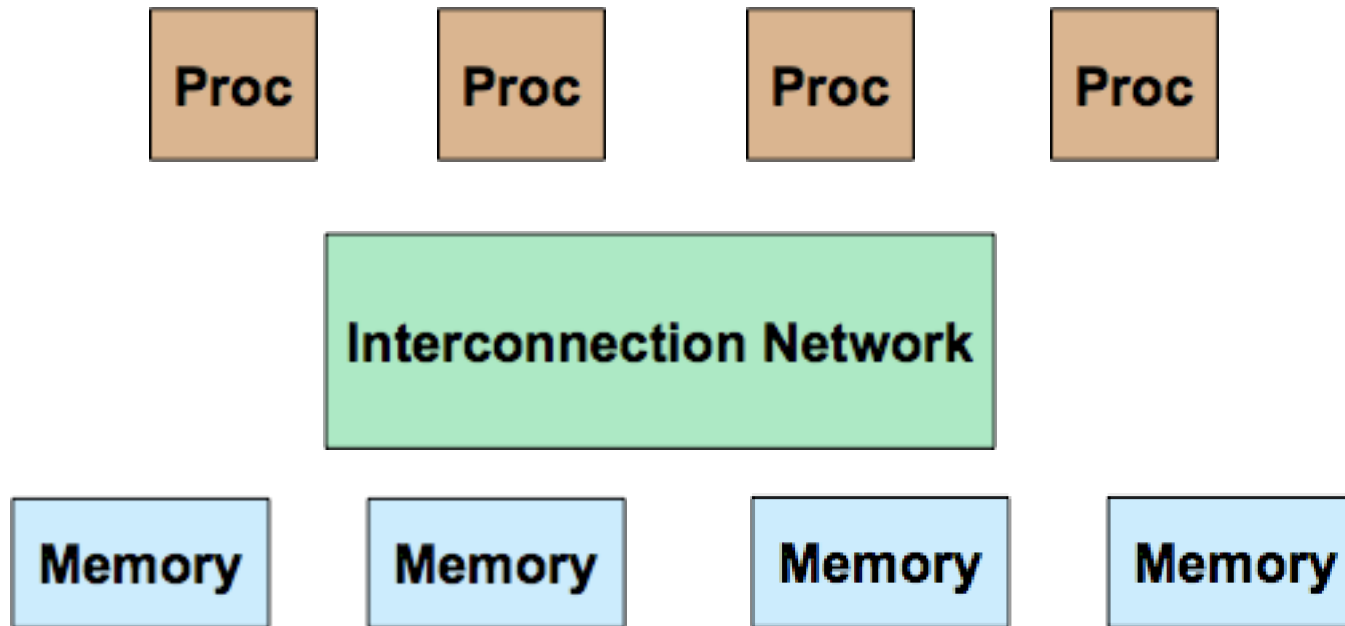


Lecture 2: Parallel Architectures

Beakal Gizachew Assefa

An Abstract View of Architecture



- How is parallelism managed?
- Where is the memory physically located?
- Is it connected directly to processors?
- What is the connectivity of the network?

Outline

- Von Neumann Architecture
 - Modifications to Von Neumann Architecture
 - Cache Hierarchy
 - Instruction Level Parallelism
 - Taxonomy of Parallel Computers
 - Memory Models
-
- Readings from Book
 - Chapter 2.1–2.4

Diversity of Architectures

- There is no canonical parallel computer – a diversity of parallel architectures
 - Hence, there is no canonical parallel programming language
- Architecture has an enormous impact on performance
 - And we wouldn't write parallel code if we didn't care about performance
- Many parallel architectures fail to succeed commercially
 - Can't always tell what is going to be around in N years

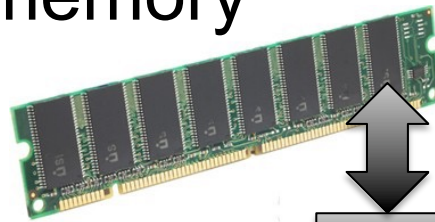
Challenge is to write parallel code that

- abstracts away architectural features,
- focuses on their commonality, and
- easily ported from one platform to another.

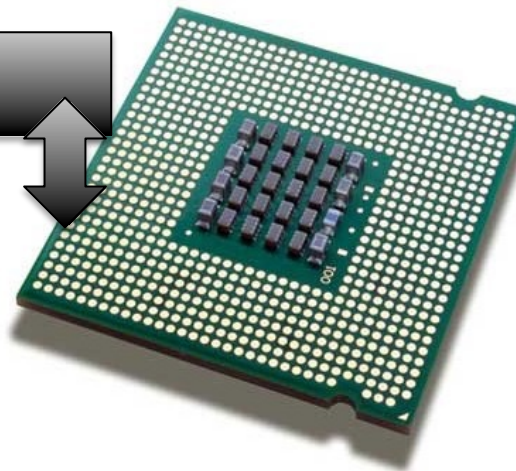
The von Neumann Architecture

- Conceptually, a von Neumann architecture executes one instruction at a time

memory

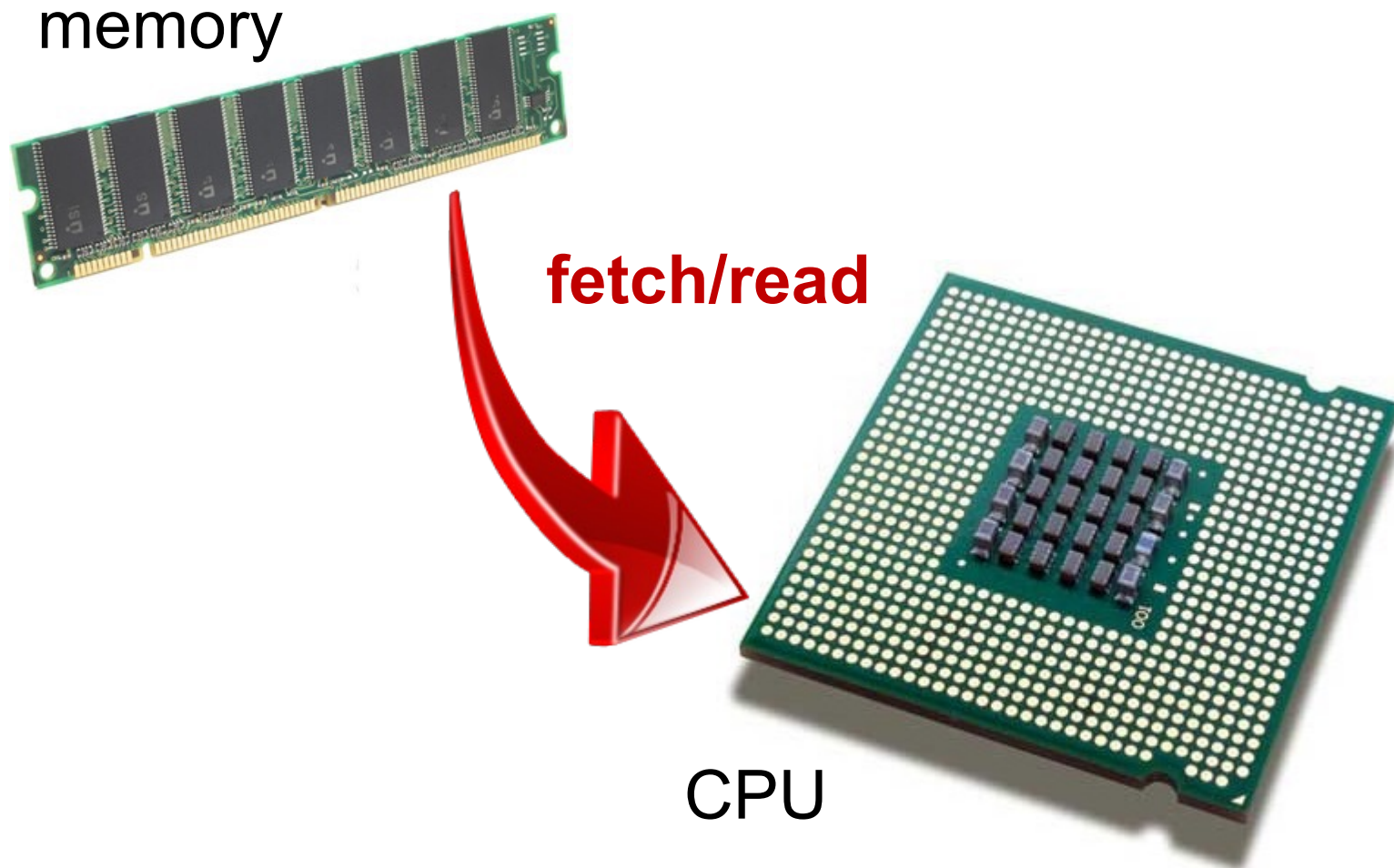


interconnect

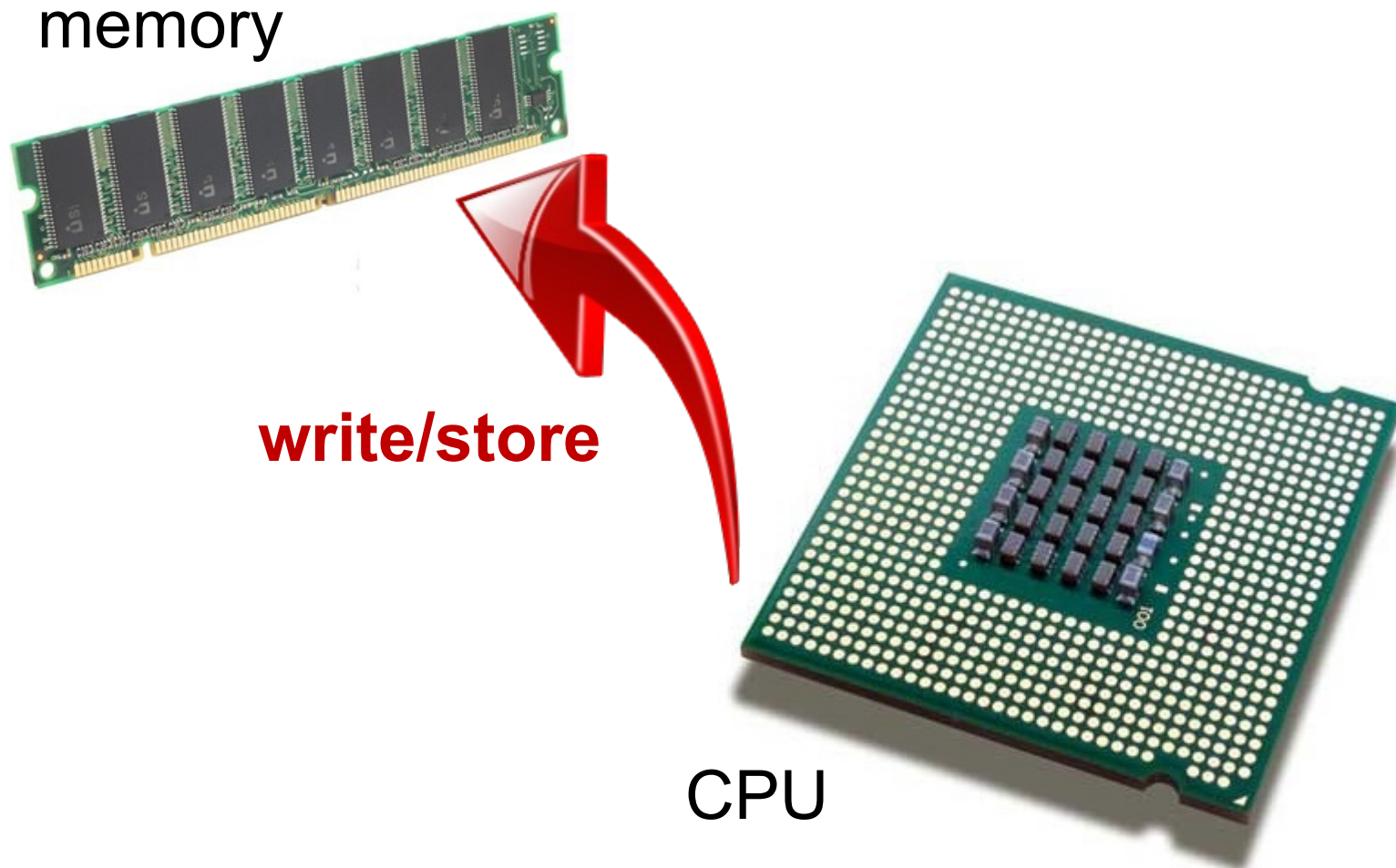


CPU

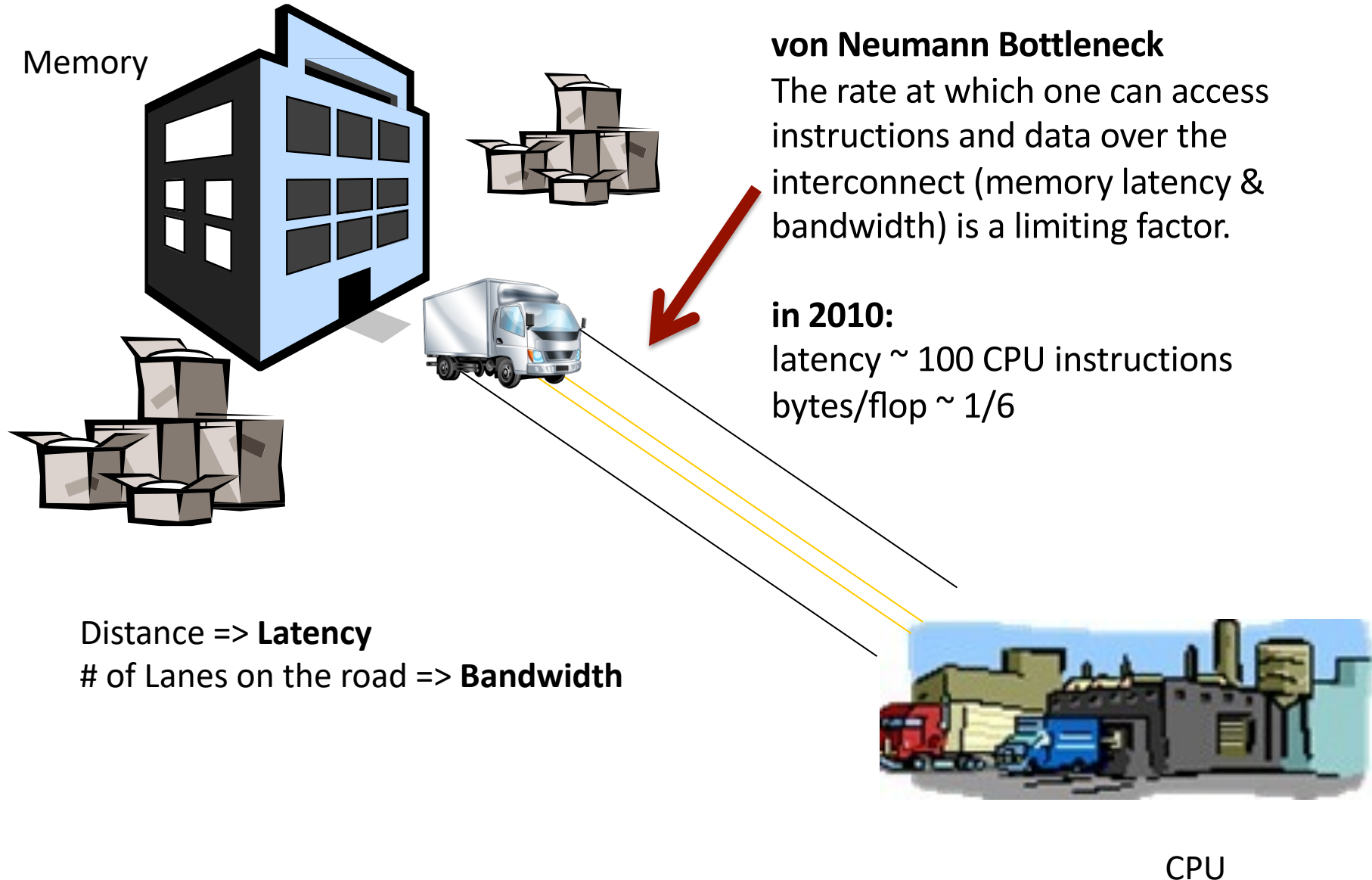
The von Neumann Architecture



The von Neumann Architecture



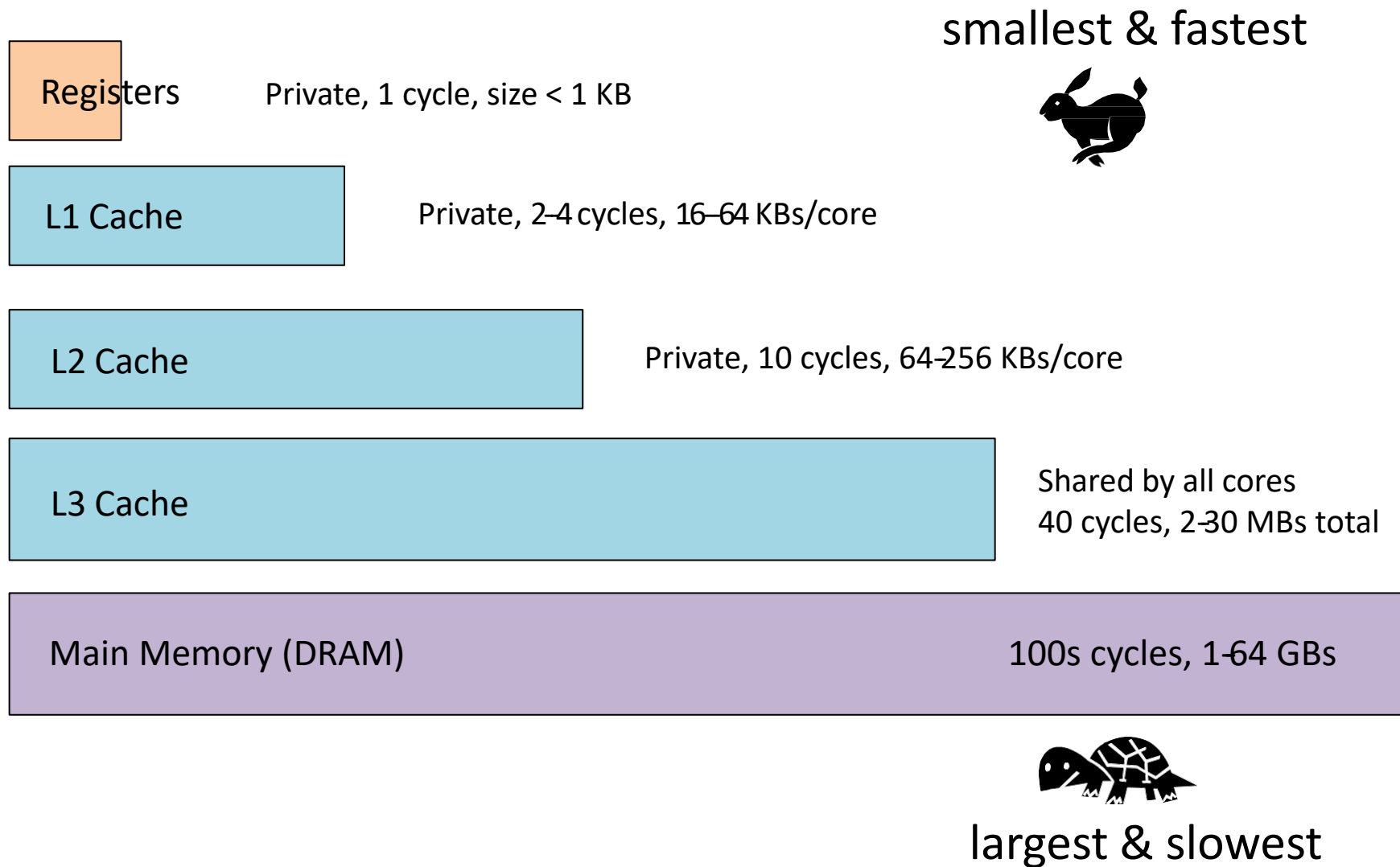
The von Neumann Bottleneck



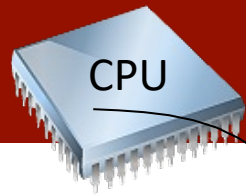
Modifications to the von Neumann Arch.

- **Cache:** fast on-chip memory
 - Works based on the **principle of locality**
 - *After accessing one memory location, a program will typically access a nearby location in the near future.*
 - Spatial Locality (nearby location)
 - Temporal Locality (same location)

Memory Hierarchy



Cache Hit



Registers

Read x

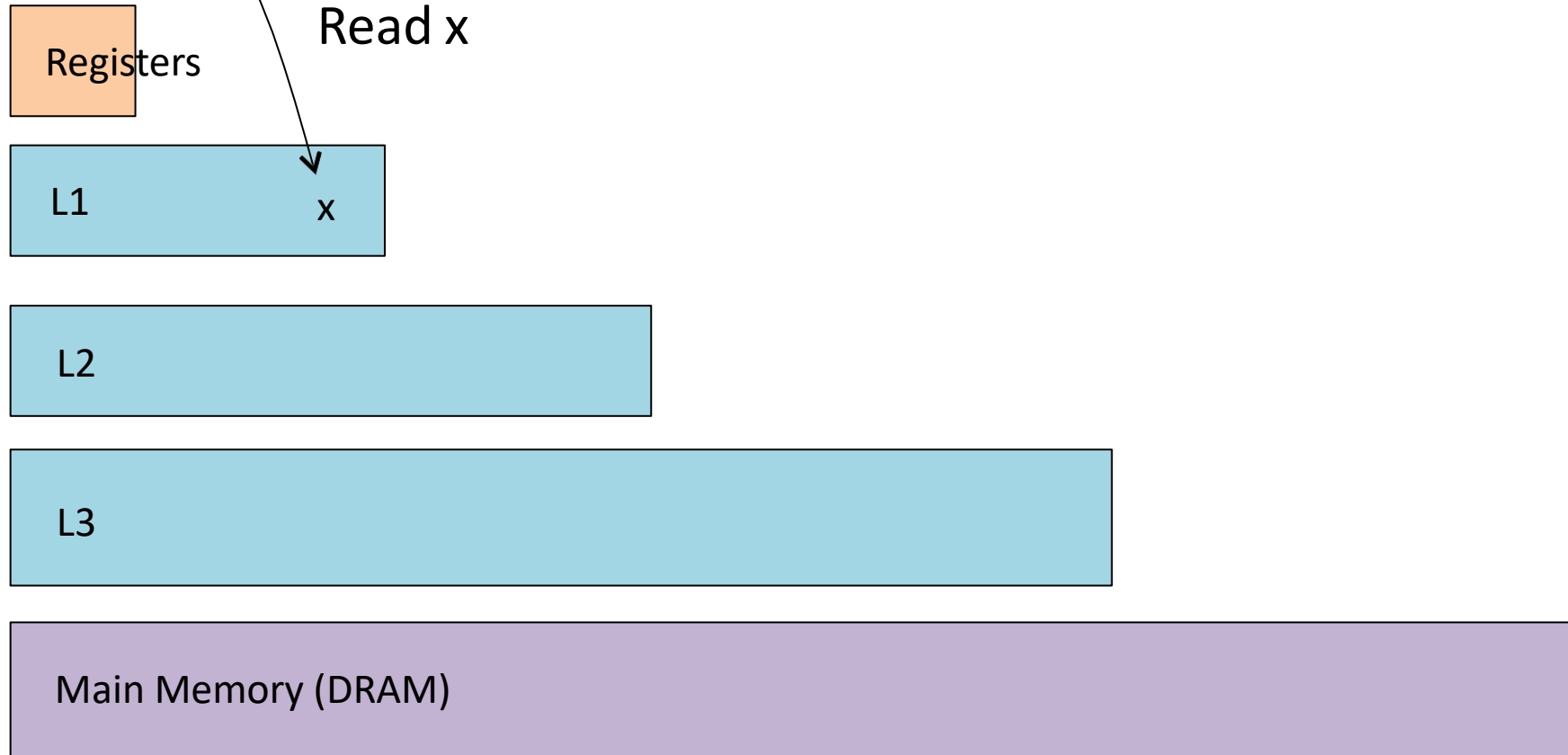
L1

x

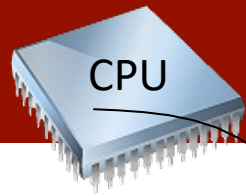
L2

L3

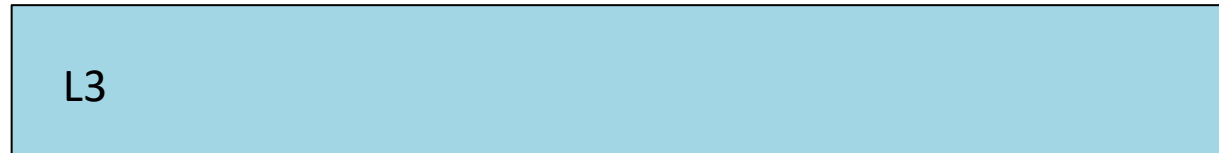
Main Memory (DRAM)



Cache Miss



Read x



- **Types of Misses**

- Capacity Misses
- Conflict Misses
- Compulsory Misses

Principle of Locality

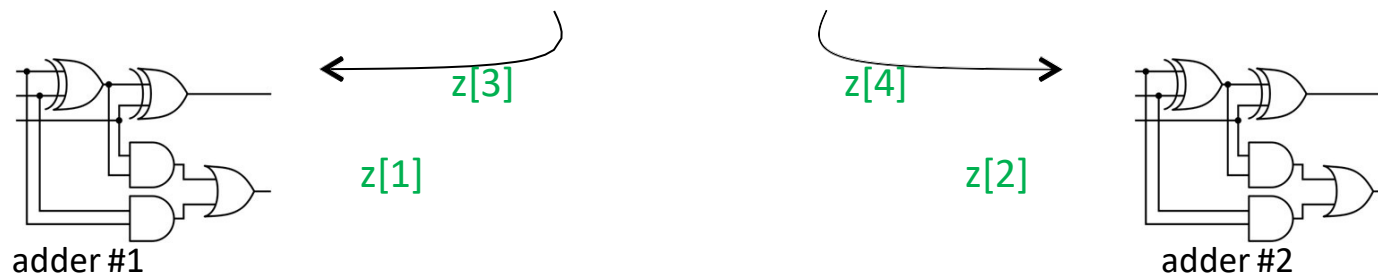
```
float *Z;  
.  
.  
Sum = 0.0;  
For (I = 0 ; I < N; I++)  
    sum += Z[I];
```

- Z[0] through Z[N-1] are stored contiguously in memory
- Float is 4 bytes (double is 8 bytes)
- Typical cache line size (block size) is 64–128 bytes
- When Z[0] is accessed, Z[1] through Z[15] are also fetched into cache

Parallelism in HW

- Instruction Level Parallelism
 - Pipelining
 - Break instructions into smaller units and overlap their executions
 - Intel Core Microarchitecture has 14–19 stage pipeline
 - Multiple issue
 - Multiple instructions can be simultaneously initiated

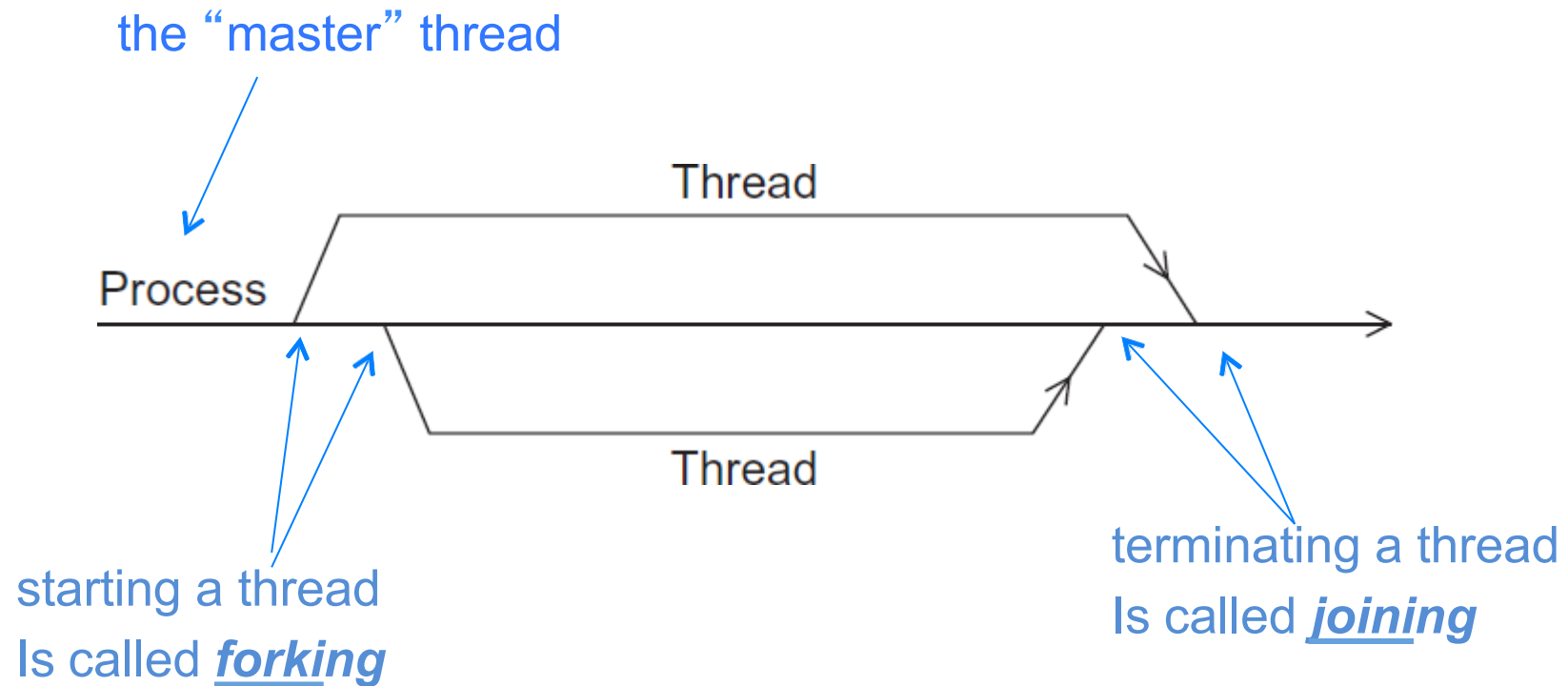
```
for (i = 0; i < 1000; i++)  
    z[i] = x[i] + y[i];
```



Process vs Thread

- Process have **their own**
 - Executables
 - Address spaces
 - System resources
 - Unique process ids
- Threads of a process **share**
 - Executable
 - Address space
 - System resources
- Threads have **different**
 - Program counters
 - Call Stacks
 - Register Files

A process and Two Threads



Hardware Multithreading

- ILP can be hard to exploit
- Programmers may expose explicit parallelism through threading
- Hardware support includes:
 - Thread private program counters, call stacks and registers
 - Requires duplication of some of hw resources
- **Simultaneous multithreading (SMT):** instructions from multiple threads are issued at each cycle (ex: Intel Hyper--Threading)

Flynn's Taxonomy

		Data Streams	
		Single	Multiple
Instruction Streams	Single	SISD: Intel Pentium 4	SIMD: SSE instructions of x86
	Multiple	MISD:	MIMD: Intel Xeon e5345

- SISD: Uniprocessor, Classic von Neumann
- SIMD: Single instruction, multiple data streams
 - Today's machines are all SIMD

SIMD

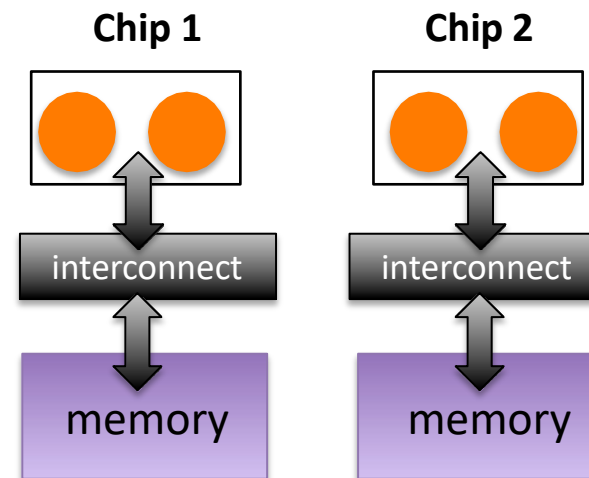
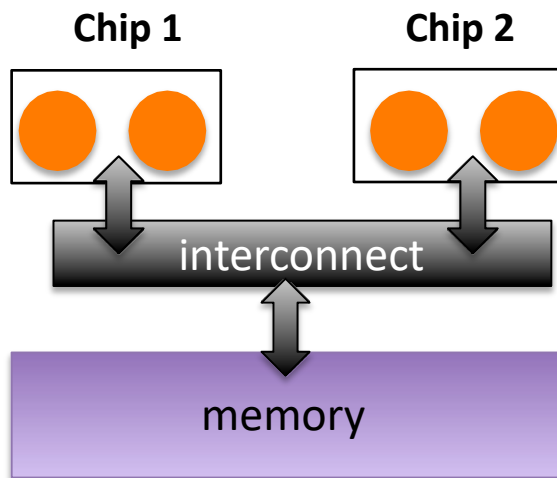
- Parallelism achieved by dividing data among the processors
- Applies the same instruction (operation) to multiple data items
 - AKA **data parallelism**
- SIMD architecture examples:
 - GPUs, CPUs with SIMD units

```
for (i = 0; i < n; i++)  
    x[i] += y[i];
```

Round3	ALU ₁	ALU ₂	ALU ₃	ALU ₄
1	X[0]	X[1]	X[2]	X[3]
2	X[4]	X[5]	X[6]	X[7]
3	X[8]	X[9]	X[10]	X[11]
4	X[12]	X[13]	X[14]	

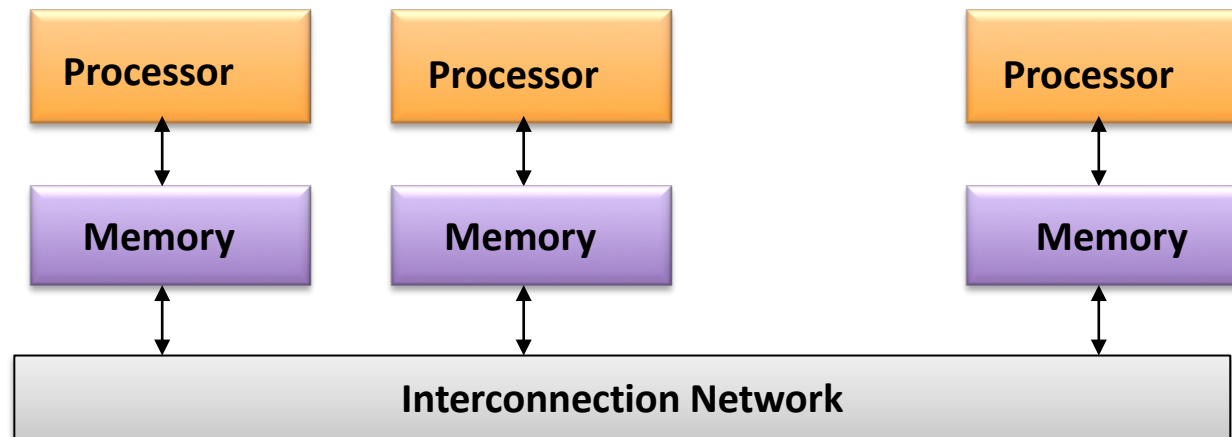
Shared Memory Architectures

- One or more (multi-core) processors
- Supports a global/single address space where each processor can access each memory location
 - Synchronize shared variables using locks
 - Memory access time
 - UMA (uniform) vs. NUMA (nonuniform) memory access



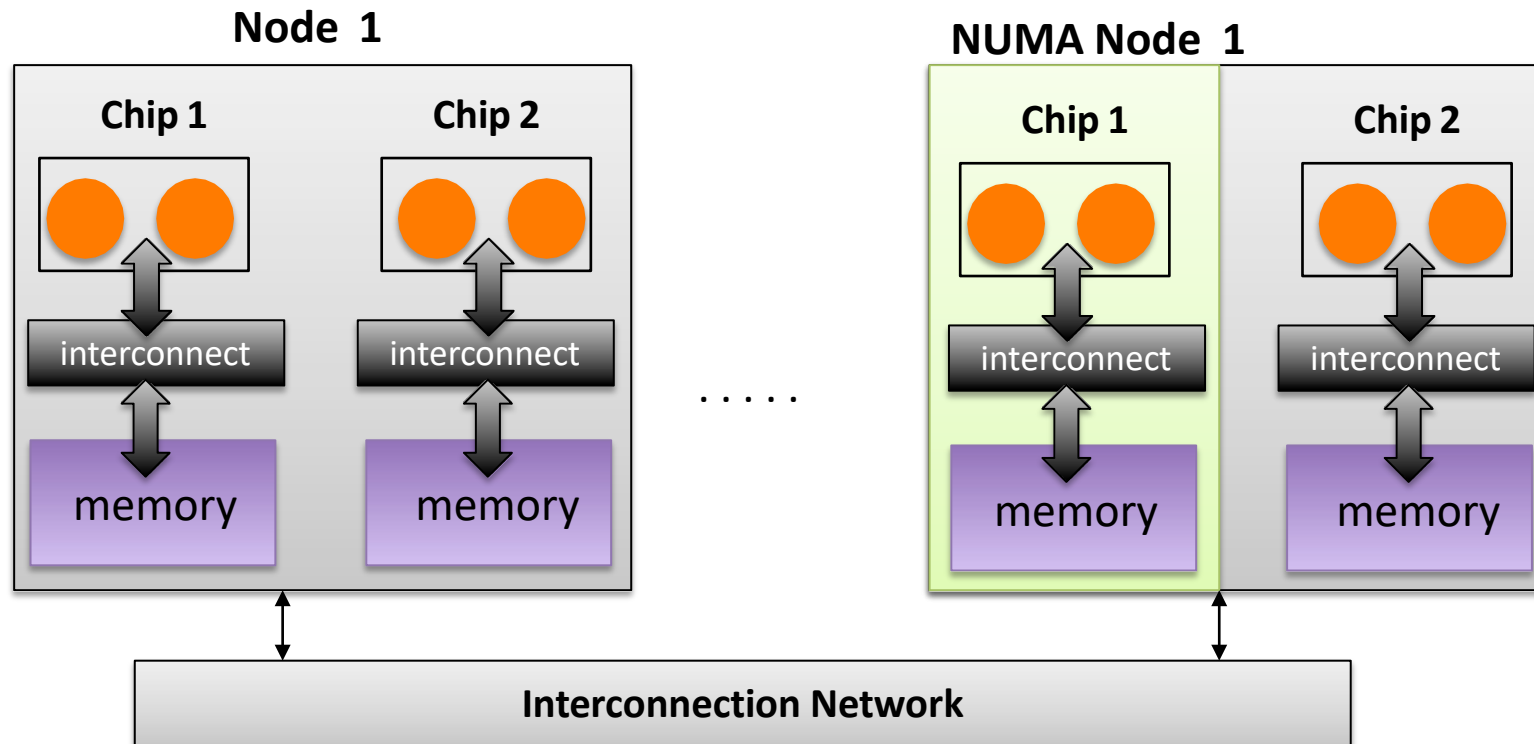
Message Passing Multiprocessors (MPP)

- Also known as distributed memory architectures
- Each processor has its own private address space
 - Processors share data by *explicitly* sending and receiving information (**message passing**)
 - Coordination is built into message passing primitives (**message send** and **message receive**)

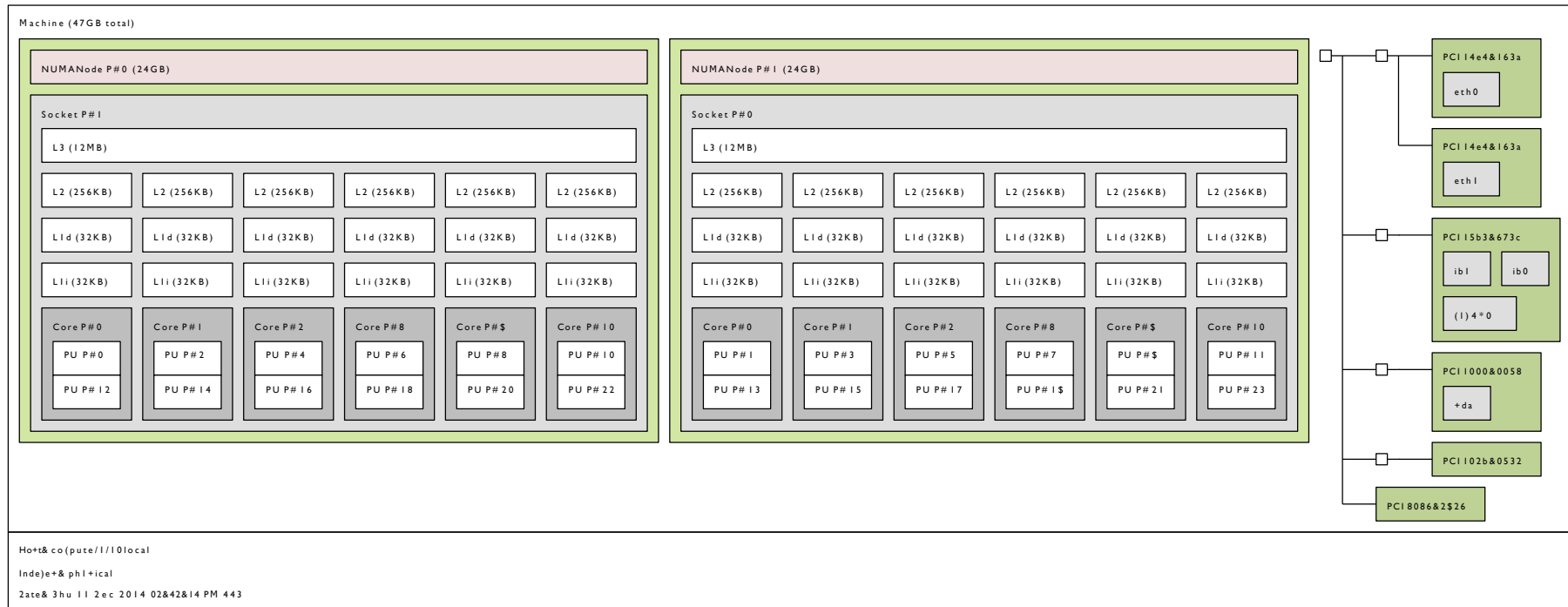


Modern Clusters and Distributed Systems

- They are both distributed memory architectures
 - Composed of nodes
 - Each node is a shared memory system
 - Which may contain multiple NUMA nodes
- Clusters, supercomputers
 - Between nodes, there is a fast interconnect

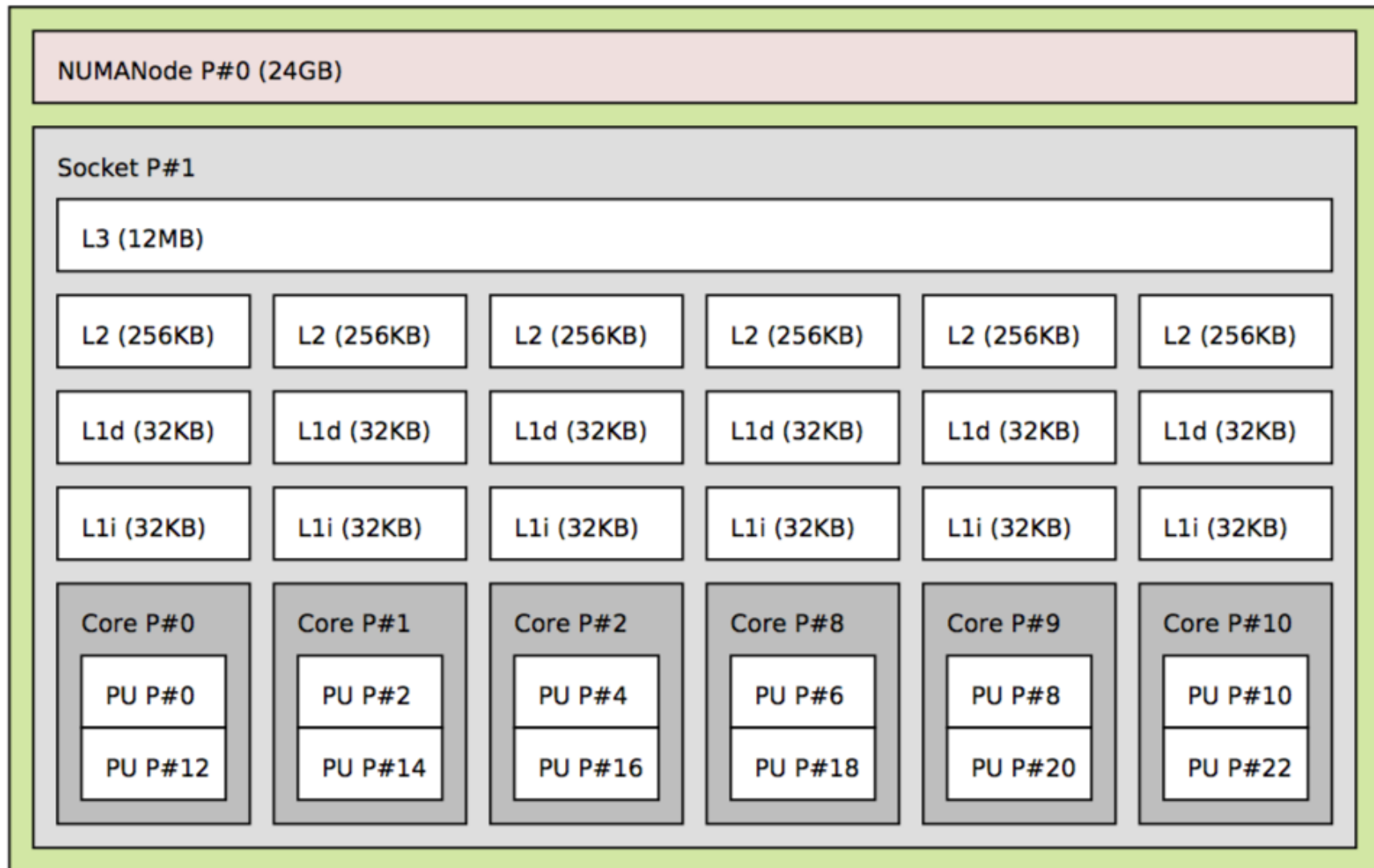


Example Cluster: Iufer

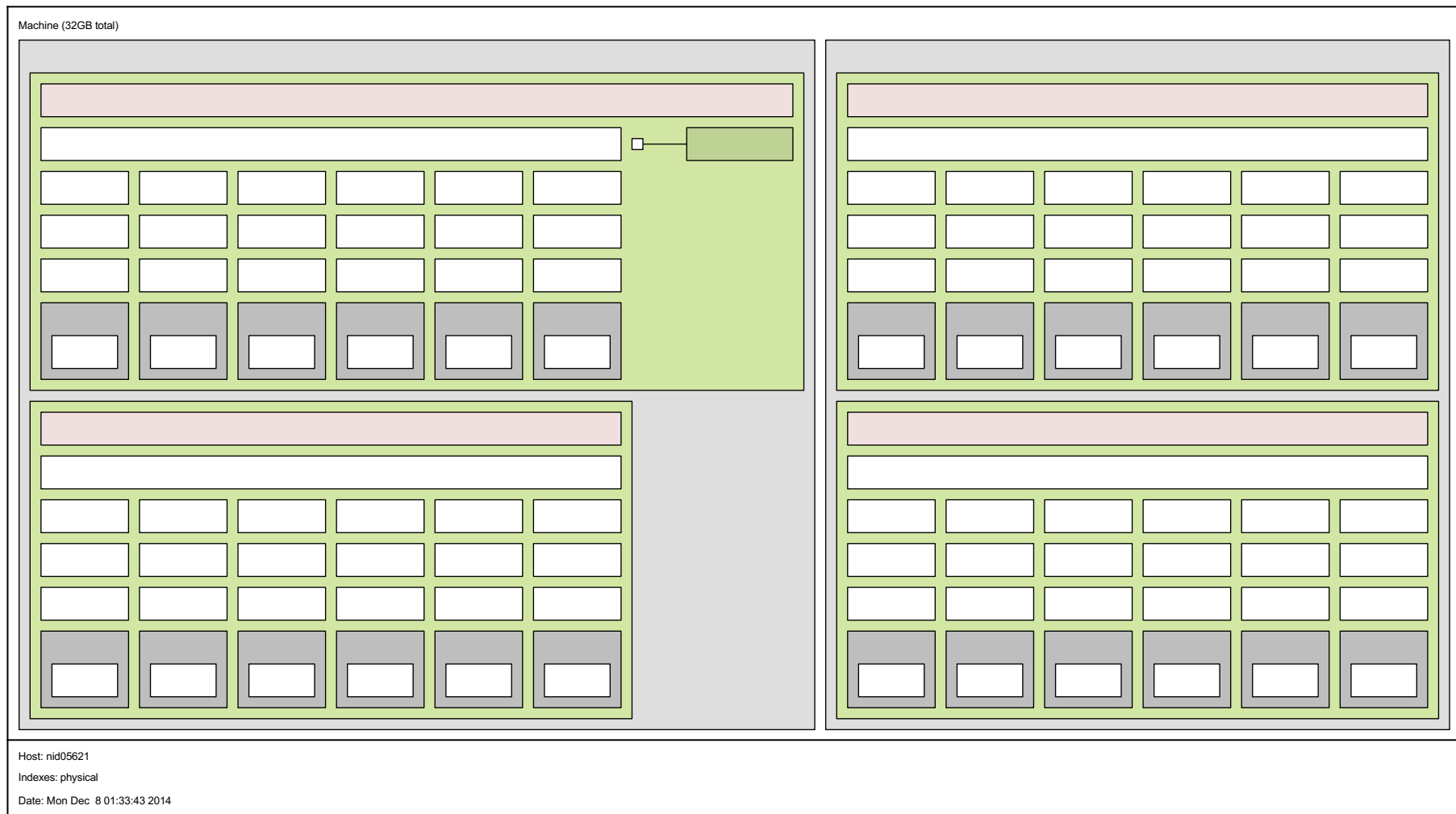


Lufer NUMA Node

Machine (47GB total)



Example: Hopper (in California)



Next

- Performance
 - Parallel Speedup and Efficiency
 - Scalability
 - Taking timings
- Multithreaded Programming
 - Starting with pthreads and OpenMP

Acknowledgments

- These slides are inspired and partly adapted from
 - Didem Unat (Koç University)
 - Mary Hall (Univ. of Utah)
 - John Shalf (LBNL)
 - Rich Vuduc (Georgia Tech)
 - Metin Aktulga (Michigan State Univ.)
 - Scott Baden (UCSD)
 - The course book (Pacheco)